

High-Throughput Event Ingestion Service

Stack	.NET 10 (C#). Any editor/IDE. AI.
Submission	Git repository (link) with full commit history
Focus areas	Performance-oriented code · Concurrency · Design rationale

1. Context

You are building the ingestion tier for a fleet of IoT devices. Each device periodically emits batches of readings. Downstream consumers — dashboards and alerting systems — need near-real-time rolling aggregates for every device, so the ingestion service must accept large batches quickly and keep per-device statistics continuously up to date.

Your task is to build the service that sits at this front door: it must absorb high-volume writes efficiently and answer aggregate queries cheaply, all in-process.

2. Objective

Build an HTTP API in .NET 10 that ingests batches of device readings and maintains rolling aggregates — count, minimum, maximum, and average — per device over a sliding time window of the last 5 minutes. All state is held in memory; no external datastore is required.

3. Functional Requirements

3.1 Ingestion endpoint

1. Accept a POST request carrying a batch of up to **50,000 readings** per call, as a JSON array.
2. Process the payload without materializing the entire request body wastefully — think carefully about how you read and deserialize the stream.
3. Update the affected per-device aggregates as readings are ingested.
4. Return promptly with a result indicating how many readings were accepted.

A single reading has the following shape:

Field	Type	Notes
deviceId	string	Up to ~10,000 distinct devices
timestamp	ISO-8601 UTC	May arrive slightly out of order
value	double	Sensor reading

3.2 Query endpoint

5. Expose a GET endpoint that, given a `deviceId`, returns the current aggregates for that device over the active window: count, min, max, and average.

6. Return an appropriate response when the device is unknown or has no readings within the window.

3.3 Windowing

- Aggregates reflect only readings whose timestamp falls within the last 5 minutes relative to the current time.
- Readings that age out of the window must stop contributing to the aggregates. The window advances continuously, not in fixed buckets, unless you justify a bucketed approximation in your design notes.
- Readings may arrive slightly out of order; handle this sensibly.

4. Non-Functional Requirements

4.1 Performance

- **The ingestion path is the hot path.** Minimize allocations and avoid unnecessary copying of the payload.
- Demonstrate deliberate use of allocation-conscious techniques where they are warranted — for example streaming JSON reading, `Span<T>` / `Memory<T>`, buffer or object pooling. Do not over-engineer; apply them where they earn their keep and be ready to justify each.

4.2 Concurrency

- Multiple ingestion requests may arrive concurrently, including for overlapping devices. Aggregation must be thread-safe with no lost updates or torn reads.
- Query requests may run concurrently with ingestion and must not observe corrupt state.
- Prefer fine-grained coordination over a single global lock; be prepared to defend your choice of synchronization primitives.

4.3 Correctness & resource use

- Memory must not grow unbounded as readings age out.
- The service should run as a single process instance for this exercise.

5. Deliverables

1. A working .NET 10 solution with clear build and run instructions (README).
2. A design document covering: the data structures chosen and why; how allocations were minimized on the ingestion path; the concurrency model and why it is correct; the windowing approach and its tradeoffs; and what you would change to scale beyond a single instance.
3. A benchmark (BenchmarkDotNet or equivalent) demonstrating ingestion throughput and allocations, with a short note on the results.
4. Automated tests covering windowing behavior and concurrent ingestion correctness.

6. Evaluation Rubric

The written rationale carries as much weight as the code. We are more interested in the quality of your tradeoff reasoning than in a maximally clever implementation.

Criterion	Weight	What we look for
Hot-path performance	30%	Streaming deserialization, minimal allocations, pooling, Span/Memory where warranted.
Concurrency correctness	25%	Thread-safe aggregation under overlapping writes; no data races or lost updates.
Windowing correctness	15%	Data ages out accurately; boundary conditions handled.
Design rationale	20%	Clear tradeoff reasoning; credible single-instance-to-scale-out story.
Tests & measurement	10%	Meaningful tests plus a benchmark backing the performance claims.

7. Out of Scope

To keep the exercise within the time budget, the following are explicitly not required. Do not spend time on them:

- Authentication, authorization, and rate limiting.
- Persistence, durability, or recovery after restart.
- Deployment, containerization, and CI pipelines.
- Any user interface.
- Horizontal scaling or distributed coordination — discuss it in the design doc instead of building it.

8. Submission & Assumptions

- Submit as a Git repository with intact commit history so we can see how the solution evolved.
- Where the specification is deliberately open, make a reasonable decision and record the assumption in your README or design doc. Surfacing and defending these choices is part of what we evaluate.
- Budget your time: a correct, well-reasoned core beats a broad but shaky solution. If you run short, note in the README what you would do next and why.

Good luck — we are looking forward to reading your rationale as much as your code.